DESENVOLVIMENTO MOBILE · MÓDULO 15 · TRILHA FLUTTER · NÍVEL INTERMEDIÁRIO

Publicação: Google Play e App Store

Identidade do app, builds de release, assinatura, Play Console, App Store Connect, TestFlight, revisão, automação com fastlane e operação depois do lançamento

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar o caminho do código até o aparelho do usuário e o papel da assinatura digital.
- Configurar nome, ícone, *splash*, versão e ambientes de um app Flutter.
- Gerar builds de release ofuscados para Android e iOS.
- Publicar na Google Play passando pelas faixas de teste.
- Publicar na App Store passando pelo TestFlight e pela revisão.
- Automatizar a entrega com fastlane e operar o app depois do lançamento.

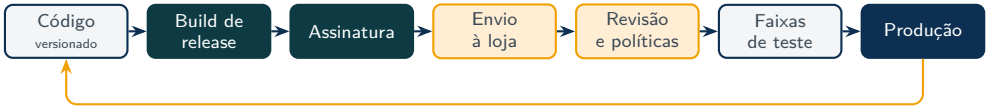
Roteiro

Neste módulo

- Fundamentos da distribuição de apps
- Preparando o app Flutter
- Google Play
- App Store
- Automação e operação
- Fechamento

Fundamentos da distribuição de apps

Do código ao aparelho do usuário



	Debug	Release
Objetivo	desenvolver	distribuir
Código	com verificações e depuração	otimizado, reduzido e ofuscável
Desempenho	mais lento	o real
Assinatura	chave de depuração automática	sua chave de release

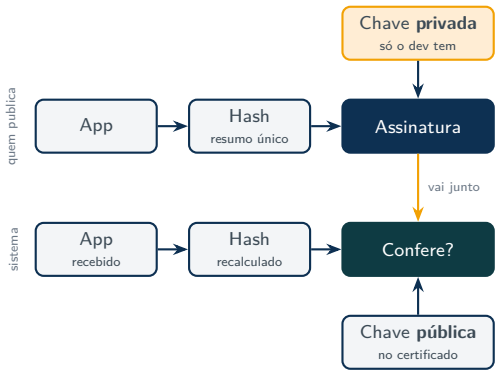
Build de release

Versão do app compilada para o usuário final: sem ferramentas de depuração, com código otimizado e assinada com a chave do desenvolvedor.

Atenção

Teste sempre o build de release antes de publicar: ofuscação e otimizações podem expor defeitos que não aparecem em debug.

Assinatura digital: a teoria



Assinatura digital

Resumo (*hash*) do pacote cifrado com a **chave privada** do desenvolvedor. Qualquer um verifica com a **chave pública**, mas só quem tem a privada consegue assinar.

O que ela garante

Integridade: o pacote não foi alterado.
Autenticidade: a atualização vem do mesmo autor do app instalado.

Analogia

O lacre de cera com o sinete do remetente: qualquer um reconhece o selo, ninguém o falsifica sem o anel.

Assinatura em cada plataforma

Android

O pacote é assinado com uma chave guardada num *keystore*. Com o **Play App Signing**, você assina com a **chave de upload**; o Google confere e reassina com a **chave do app**, que fica guardada com ele.



Dica

Perdeu a chave de upload? Com o Play App Signing, o Google permite trocá-la. Sem ele, perder a chave impede atualizar o app.

iOS

Todo app precisa ser assinado para rodar num iPhone. Três peças: o **certificado** (sua identidade), o **App ID** e o **perfil de provisionamento**, que liga certificado, app, capacidades e aparelhos.

Dica

O Xcode gerencia tudo isso com *Automatically manage signing*.

Atenção

Chaves e certificados são segredos: nunca no Git; guarde cópias num cofre de senhas.

Versionamento

Versionamento semântico (SemVer)

MAJOR.MINOR.PATCH: **MAJOR** muda com quebras de compatibilidade, **MINOR** com funcionalidades novas compatíveis e **PATCH** com correções.



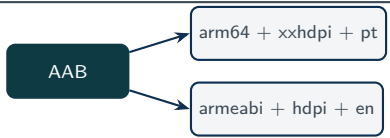
	Para pessoas	Para a loja
Android	`versionName`	`versionCode`
iOS	`CFBundleShort-VersionString`	`CFBundleVersion`
Flutter	`version: 2.4.1+27`	(o número após o `+`)

Regra das lojas

O número do build **sempre aumenta**. Não existe "voltar versão": para desfazer um problema, publica-se uma versão nova com número maior.

Formatos de pacote e tamanho do app

Formato	O que é
APK	pacote Android instalável, com tudo para todos os aparelhos
AAB	*Android App Bundle*: formato de **envio** à Play, que gera APKs sob medida para cada aparelho
IPA	pacote iOS enviado à App Store; a Apple também entrega variantes por aparelho (*App Thinning*)



Por que o tamanho importa

Apps grandes demoram a baixar, ocupam espaço e perdem instalações, principalmente em planos de dados limitados.

Como os builds encolhem

Tree shaking e ***shrinking*** removem código não usado; a **ofuscação** troca nomes por símbolos curtos; recursos são comprimidos e separados por aparelho.

Atenção

Código ofuscado gera relatórios de falha ilegíveis. Guarde os **símbolos de depuração** de cada versão para decodificá-los.

As duas grandes lojas

	Google Play	App Store
Conta	taxa única de US$ 25; pessoal ou organização	Apple Developer Program, US$ 99 por ano (isenção para instituições de ensino e sem fins lucrativos)
Revisão	automatizada e humana; de horas a dias (contas novas esperam mais)	humana (*App Review*); em geral de um a dois dias
Testes	faixas interna, fechada e aberta	TestFlight interno (até 100) e externo (até 10 000)
Lançamento gradual	percentual definido por você	*phased release* automático em 7 dias
Requisito anual	`targetSdk` 36 desde 31/08/2026	Xcode 26 e SDK do iOS 26 desde 28/04/2026
Comissão	de 15 % a 30 % sobre vendas digitais, conforme receita, produto e país	idem

Dica

Regras de comissão e de pagamento externo mudam com leis e decisões judiciais (União Europeia, EUA, Japão, Coreia). Confira antes de definir o modelo de negócio (Módulo 20).

Privacidade desde a concepção e LGPD

LGPD (Lei 13.709/2018)

Regula o tratamento de **dados pessoais** no Brasil: tudo que identifica ou pode identificar uma pessoa (nome, e-mail, matrícula, localização, identificador do aparelho).

Princípios que mais afetam um app

Finalidade (para quê), **necessidade** (o mínimo), **transparência** (explicar), **segurança** (proteger) e uma **base legal** para cada uso (consentimento, execução de contrato...).

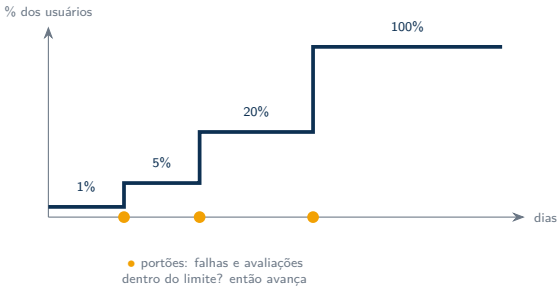
O que as lojas exigem

- Política de privacidade publicada.
- Declaração de dados coletados: *Data safety* (Play) e *App Privacy* (App Store).
- Textos de uso para cada permissão.
- **Exclusão de conta** dentro do app, se ele permite criar conta.

Atenção

A declaração deve incluir o que os SDKs de terceiros coletam (analytics, anúncios, Crashlytics). Não é assessoria jurídica: consulte o encarregado de dados (DPO) da instituição.

Gestão de lançamentos



Lançamento gradual

Liberar a versão para uma fração crescente dos usuários, avançando só se os indicadores de saúde continuarem bons.

Ferramentas do processo

- **Trem de versões**: lançar em datas fixas (a cada duas semanas, por exemplo).
- ***Feature flags***: ligar recursos remotamente, sem nova versão.
- **Versão mínima**: obrigar a atualização quando a antiga é insegura.
- **Notas de versão** claras para usuários e revisores.

ASO: o funil da loja



ASO — *App Store Optimization*

Conjunto de práticas para o app ser **encontrado** na busca da loja e **escolhido** por quem vê a ficha.

Etapa	O que influencia
Ser encontrado	título, subtítulo ou descrição curta, palavras-chave, categoria
Ser escolhido	ícone, capturas de tela, vídeo, nota e avaliações
Ser mantido	qualidade, desempenho, respostas às avaliações

Dica

As duas lojas permitem testes A/B da ficha (ícone, capturas, textos).

Depois do lançamento: saúde do app

Indicador	O que mede
Usuários sem falhas	% de usuários que não tiveram nenhum *crash* no período
Taxa de ANR	% de sessões em que a tela travou (Android)
Tempo de inicialização	do toque no ícone à primeira tela útil
Retenção D1/D7/D30	% de usuários que voltam 1, 7 e 30 dias após instalar
Nota e avaliações	percepção dos usuários; comentários apontam defeitos

Limites do Android vitals

Acima de 1,09 % de falhas percebidas ou 0,47 % de ANRs percebidos, o app é considerado de "mau comportamento" e perde visibilidade na Play.

Ciclo contínuo

Monitorar → priorizar → corrigir → lançar gradualmente → monitorar. Falhas e avaliações alimentam o *backlog* da próxima versão.

Dica

Responda às avaliações com educação e objetividade: a resposta é pública e mostra que o app é cuidado.

Preparando o app Flutter

Identidade: nome, ícone, *splash* e versão

pubspec.yaml

```yaml
name: labmonitor
version: 1.2.0+12        # versão + número do build

flutter_launcher_icons:
  android: true
  ios: true
  image_path: assets/icone/labmonitor.png
  adaptive_icon_background: "#0D3052"
  adaptive_icon_foreground: assets/icone/frente.png

flutter_native_splash:
  color: "#0D3052"
  image: assets/icone/splash.png
```

```
$ flutter pub add -d flutter_launcher_icons \
    flutter_native_splash
$ dart run flutter_launcher_icons
$ dart run flutter_native_splash:create
```

Item	Onde fica
ID Android	`applicationId` em `android/app/build.gradle.kts`
ID iOS	*Bundle Identifier* no alvo `Runner` do Xcode
Nome exibido	`android:label` (manifesto) e `CFBundleDisplayName` (`Info.plist`)
Versão	`version` do `pubspec.yaml`

Atenção

O identificador (`br.edu.ifma.labmonitor`) é **permanente** depois da primeira publicação. Escolha com cuidado.

Modos de compilação e builds de release

Modo	Características
debug	compilação JIT, *hot reload*, `asserts` ativos, DevTools completo
profile	AOT como o release, com medição de desempenho
release	AOT, *tree shaking*, sem `asserts` nem depuração

Por que o release é mais rápido

Em release, o Dart é compilado **antes** (AOT) para código de máquina ARM; em debug, roda numa VM com compilação sob demanda (JIT), que permite o *hot reload*.

```
# Android: App Bundle para a Play
$ flutter build appbundle --release \
    --obfuscate --split-debug-info=build/simbolos

# iOS: pacote para a App Store (no Mac)
$ flutter build ipa --release \
    --obfuscate --split-debug-info=build/simbolos

# o que ocupa espaço no app?
$ flutter build appbundle --analyze-size
```

Atenção

Guarde a pasta `build/simbolos` de **cada** versão e envie-a ao Crashlytics (`firebase crashlytics:symbols:upload`); sem ela, as pilhas de erro ficam ilegíveis.

Ambientes: desenvolvimento, homologação e produção

env/producao.json

```
{
  "API_URL": "https://api.labmonitor.dev/v1",
  "AMBIENTE": "producao"
}
```

lib/config.dart

```
const apiUrl = String.fromEnvironment('API_URL',
    defaultValue: 'http://10.0.2.2:8080/v1');
const ambiente = String.fromEnvironment('AMBIENTE',
    defaultValue: 'dev');
```

```
$ flutter run --dart-define-from-file=env/dev.json
$ flutter build appbundle \
    --dart-define-from-file=env/producao.json
```

Mesmo código, configurações diferentes

A URL da API, o projeto Firebase e o nível de log mudam por ambiente. O valor é decidido na hora do build, não no código.

Flavors

Para ter "LabMonitor Homolog" e "LabMonitor" instalados lado a lado, use *flavors* (`productFlavors` no Android, *schemes* no iOS) e `-flavor homolog`.

Segredo no binário

Valores de `-dart-define` ficam gravados no app e podem ser extraídos.

Google Play

Chave de upload e assinatura no Gradle

android/app/build.gradle.kts (trechos)

```
val props = Properties().apply {
    val f = rootProject.file("key.properties")
    if (f.exists()) load(FileInputStream(f))
}

android {
    signingConfigs {
        create("release") {
            keyAlias = props["keyAlias"] as String
            keyPassword = props["keyPassword"] as String
            storeFile = file(props["storeFile"] as String)
            storePassword = props["storePassword"] as String
        }
    }
    buildTypes {
        release {
            signingConfig = signingConfigs.getByName("release")
        }
    }
}
```

```
$ keytool -genkey -v -keystore \
    ~/chaves/upload.jks \
    -keyalg RSA -keysize 2048 \
    -validity 10000 -alias upload
```

android/key.properties

```
storePassword=********
keyPassword=********
keyAlias=upload
storeFile=/home/aluno/chaves/upload.jks
```

Atenção

`key.properties` e `*.jks` vão para o `.gitignore`. No CI, use segredos do repositório.

Play Console: do cadastro à produção

Antes do primeiro envio

1. Criar o app (nome, idioma, gratuito ou pago).
2. Ficha da loja: título (até 30 caracteres), descrição curta (80), descrição completa, ícone 512×512, destaque 1024×500, capturas.
3. Classificação indicativa (questionário IARC).
4. Segurança dos dados, público-alvo, anúncios.
5. Acesso ao app: credenciais de teste para os revisores.
6. Enviar o AAB às faixas: **interna** (até 100 pessoas) → **fechada** → **aberta** → **produção**.

Regras atuais

- `targetSdk` 36 ou maior desde 31/08/2026 (confira em `build.gradle.kts`).
- Contas **pessoais** criadas após 13/11/2023: teste fechado com 12 testadores por 14 dias seguidos.
- Desde 30/09/2026, no Brasil, apps em aparelhos certificados precisam vir de desenvolvedores verificados, mesmo fora da Play.

App Store

Pré-requisitos da Apple

O que você precisa

- Um **Mac** com **Xcode 26** ou mais novo (exigido para envios desde 28/04/2026).
- Conta no **Apple Developer Program**.
- *Bundle ID* registrado e assinatura automática no alvo `Runner` (*Signing & Capabilities*).
- Registro do app no **App Store Connect**: nome, idioma, *bundle ID* e SKU.

Sem Mac?

Serviços de CI com máquinas macOS (Codemagic, GitHub Actions, Bitrise) geram e enviam o build iOS.

Privacidade no iOS

- Textos de uso no `Info.plist` para cada permissão (Módulo 13).
- *App Privacy* preenchido no App Store Connect.
- **Manifesto de privacidade** (`PrivacyInfo.xcprivacy`) declarando dados coletados e o motivo do uso de certas APIs; plugins trazem os seus.

Instituições

Instituições de ensino podem solicitar isenção da anuidade do programa.

Build, envio e TestFlight

```
$ flutter build ipa --release \
    --obfuscate --split-debug-info=build/simbolos
# gera build/ios/ipa/*.ipa

# envio pelo terminal (chave da API do
# App Store Connect) ou pelo app Transporter
$ xcrun altool --upload-app --type ios \
    -f build/ios/ipa/*.ipa \
    --apiKey $CHAVE --apiIssuer $EMISSOR
```

Dica

Alternativa visual: abrir `ios/Runner.xcworkspace` no Xcode, *Product* → *Archive* → *Distribute App*.

TestFlight	Como funciona
Interno	até 100 membros da equipe; sem revisão
Externo	até 10 000 pessoas por e-mail ou link público; passa por revisão beta
Validade	cada build expira em 90 dias

Envio para revisão

Escolha o build, preencha as informações para o revisor (conta de demonstração!) e envie. Aprovado, publique na hora, numa data ou com *phased release*.

App Review: motivos comuns de rejeição

Diretriz	Problema típico	Como evitar
2.1 Completude	falhas, telas vazias, *links* quebrados, conteúdo provisório	testar o build de release em aparelho real
2.3 Metadados	capturas que não mostram o app real, descrição enganosa	imagens fiéis às telas atuais
4.2 Funcionalidade mínima	app que só embrulha um site	oferecer valor de app nativo
4.8 Serviços de login	login social sem opção equivalente focada em privacidade	incluir Sign in with Apple
5.1.1 Coleta de dados	permissão sem texto claro; conta sem exclusão	textos específicos; excluir conta no app
3.1.1 Compras no app	vender conteúdo digital fora do sistema da Apple	usar compra no app onde for exigido

Dica

Se o app exige login, informe uma conta de demonstração. Responda à revisão pelo *Resolution Center*, com calma e objetividade.

Automação e operação

Automatizando a entrega com fastlane

android/fastlane/Fastfile e ios/fastlane/Fastfile

```
# Android
lane :interno do
  sh("cd ../.. && flutter build appbundle --release")
  upload_to_play_store(track: "internal",
    aab: "../build/app/outputs/bundle/release/app-release.aab")
end

# iOS
lane :beta do
  sh("cd ../.. && flutter build ipa --release")
  upload_to_testflight(
    ipa: "../build/ios/ipa/LabMonitor.ipa")  # nome do app
end
```

```
$ cd android && fastlane interno
$ cd ios && fastlane beta
```

Entrega contínua

Cada versão aprovada no CI é compilada, assinada e enviada à faixa de teste sem passos manuais. Menos erro humano, entregas frequentes.

Dica

O fastlane também gera capturas de tela, atualiza a ficha da loja e incrementa o número do build.

Dica

O Módulo 18 monta o *pipeline* completo no GitHub Actions.

Atualização obrigatória e pedido de avaliação

Na abertura do app

```dart
// versão mínima no Remote Config (Módulo 12)
final minima = Version.parse(rc.getString('versao_minima'));
final info = await PackageInfo.fromPlatform();
if (Version.parse(info.version) < minima) {
  router.go('/atualizar');          // bloqueia o uso
}

// Android: atualização pela própria Play
final upd = await InAppUpdate.checkForUpdate();
if (upd.updateAvailability ==
    UpdateAvailability.updateAvailable) {
  await InAppUpdate.performImmediateUpdate();
}

// após uma reserva bem-sucedida
final review = InAppReview.instance;
if (await review.isAvailable()) review.requestReview();
```

Pacotes

`pub_semver` compara versões; `in_app_update` usa a API de atualização da Play (só Android); `in_app_review` abre a avaliação nativa das duas lojas.

Atenção

O sistema decide se mostra o pedido de avaliação (no iOS, no máximo três vezes por ano). Nunca ofereça recompensa por nota nem peça logo na primeira abertura.

Fechamento

Resumo

Resumo

- O release é otimizado, ofuscado e assinado; a assinatura garante integridade e autoria.
- Versão para pessoas (SemVer) e número de build para a loja, sempre crescente.
- `flutter build appbundle` e `flutter build ipa` geram os pacotes; guarde os símbolos.
- Play: faixas de teste, regras de `targetSdk`, testadores e verificação de desenvolvedor.
- App Store: Xcode 26, TestFlight, App Review e declarações de privacidade.
- Lançamento gradual, monitoramento e automação fecham o ciclo.

Glossário (1/2)

Termo	Significado
Build de release	versão otimizada e assinada do app, destinada ao usuário final
Assinatura digital	*hash* do pacote cifrado com a chave privada; prova integridade e autoria
Keystore	arquivo protegido por senha que guarda as chaves de assinatura no Android
Play App Signing	serviço em que o Google guarda a chave do app e você assina com a chave de upload
Perfil de provisionamento	arquivo da Apple que liga certificado, App ID, capacidades e aparelhos
AAB	*Android App Bundle*: formato de envio do qual a Play gera APKs por aparelho
IPA	pacote do app iOS enviado ao App Store Connect
Ofuscação	troca de nomes do código por símbolos curtos, dificultando engenharia reversa

Glossário (2/2)

Termo	Significado
Símbolos de depuração	arquivos que traduzem pilhas ofuscadas de volta para nomes legíveis
SemVer	versionamento semântico: MAJOR.MINOR.PATCH
Faixa de teste	canal da Play para distribuir a grupos (interna, fechada, aberta)
TestFlight	serviço da Apple para distribuir builds de teste
App Review	revisão humana dos apps enviados à App Store
Lançamento gradual	liberação da versão para uma porcentagem crescente de usuários
ASO	*App Store Optimization*: práticas para ser encontrado e escolhido na loja
Flavor	variante do app (homologação, produção) gerada do mesmo código

Questões de revisão

1. O que a assinatura digital garante, e por que perder a chave pode impedir atualizações?
2. Qual a diferença entre a chave de upload e a chave do app no Play App Signing?
3. Por que o número do build precisa sempre aumentar? Como "desfazer" uma versão com defeito?
4. Em que o AAB difere do APK, e que vantagem ele traz para o usuário?
5. Por que é preciso guardar os símbolos de depuração de cada versão ofuscada?
6. Por que valores passados com `-dart-define` não servem para guardar segredos?
7. Cite três motivos comuns de rejeição na App Store e como evitá-los.
8. Como um lançamento gradual reduz o risco de uma versão nova?

Exercícios

Exercício 1 — Identidade

Defina o identificador, gere ícone adaptativo e *splash* com as cores do curso e configure três ambientes com `-dart-define-from-file`.

Exercício 2 — Release Android

Crie a chave de upload, configure a assinatura, gere o AAB ofuscado e analise o tamanho com `-analyze-size`.

Exercício 3 — Ficha da loja

Escreva título, descrição curta e completa e planeje cinco capturas de tela, aplicando as ideias de ASO.

LabMonitor — Etapa 15

Publique o LabMonitor na faixa de teste fechado da Play com a turma (e no TestFlight, se houver Mac), com política de privacidade, exclusão de conta e lançamento gradual da versão 1.0.

Para aprofundar

- Publicar no Android: `https://docs.flutter.dev/deployment/android`
- Publicar no iOS: `https://docs.flutter.dev/deployment/ios`
- Ofuscação: `https://docs.flutter.dev/deployment/obfuscate`
- Play Console: `https://support.google.com/googleplay/android-developer`
- Diretrizes da App Store: `https://developer.apple.com/app-store/review/guidelines`
- fastlane: `https://docs.fastlane.tools`

Respostas das questões de revisão

1. Integridade (o pacote não foi alterado) e autenticidade (vem do mesmo autor). O sistema só aceita atualizações assinadas pela mesma chave; sem ela, não há como provar a autoria.
2. A de upload assina o que você envia e pode ser trocada; a do app, guardada pelo Google, assina os APKs entregues aos aparelhos.
3. A loja e o sistema usam o número para saber qual build é mais novo. Para desfazer, publica-se uma versão nova, com número maior, contendo a correção (ou o código anterior).
4. O APK contém tudo para todos os aparelhos; o AAB permite à Play gerar APKs só com o código, as imagens e os idiomas de cada aparelho: download menor.
5. Sem eles, as pilhas de falha mostram nomes ofuscados e não é possível localizar o defeito no código.
6. Eles são gravados no binário, que pode ser descompactado e analisado por qualquer pessoa.
7. Falhas ou telas vazias (testar o release), login social sem opção equivalente (Sign in with Apple), permissões sem texto claro ou conta sem exclusão (textos específicos e exclusão no app).
8. Um defeito atinge só a fração inicial de usuários; os indicadores de saúde decidem se o lançamento avança ou é interrompido.

Fim do Módulo 15

Próximo: Módulo 16 (comum) — Backend terceirizado: Firebase, Supabase, AWS Amplify, Appwrite e outros.